

Hier ist die Analyse des PDF-Auszugs mit den Aufgaben und Lösungen:

Aufgabe 1: Fehlersuche

** (a) Fehler: Groß-/Kleinschreibung von `True`

- **Fehler:** `bestanden = true` (Python ist case-sensitive, `True` muss großgeschrieben werden).
- **Korrektur:** `bestanden = True`

** (b) Fehler: Längeninkonsistenz zwischen Listen

- **Fehler:** `stadt` hat 4 Elemente, `einwohner` nur 3 → `IndexError` bei `stadt[3]`.
- **Korrektur:** Entweder `einwohner` um ein Element erweitern oder `stadt` kürzen.

** (c) Fehler: Kein Basisfall in rekursiver Fakultätsfunktion

- **Fehler:** Keine Abbruchbedingung für `n = 0` oder `n = 1` → unendliche Rekursion.
- **Korrektur:** `if n <= 1: return 1` hinzufügen.

** (d) Fehler: Uninitialisierte Variable `i`

- **Fehler:** `i` wird nicht definiert, bevor es in `while i <= 10:` verwendet wird.
- **Korrektur:** `i = 0` vor der Schleife initialisieren.

** (e) Fehler: Division durch 0

- **Fehler:** `b = 0.6` wird zu `int(b) = 0` → Division durch 0.
- **Korrektur:** `b` auf einen Wert > 0 setzen (z. B. `b = 1.6`).

**Aufgabe 2: Ausgabeanalyse

** (a) Ausgabe: `True`

- `vier \* vier == zwei` → `2 \* 2 == 4` → `4 == 4` → `True`.

** (b) Ausgabe: `3`

- `range(a, b)` → `range(3, 4)` → nur `3` wird ausgegeben.

** (c) Ausgabe: `i`, `j`, `k` (jeweils neuzeilig)

- Schleife durchläuft `i` mit `i` von 0 bis 2.

** (d) Ausgabe: `rekursiv`

- Indizes: `16` (r), `17` (e), `18` (k), `19` (u), `5` (r), `9` (u), `2` (e), `18` (k), `16` (r), `4` (s).

** (e) Ausgabe: `1`, `2`, `3`, `4`, `5` (jeweils neuzeilig)

- Rekursion startet bei `k(1)` und endet bei `k(5)`.

**Aufgabe 3: Run Length Encoding (RLE)

** (a) `encode(text)`

- **Lösung:

```
```python
```

```
def encode(text):
```

```
 result = []
```

```
 count = 1
```

```

for i in range(1, len(text)):
 if text[i] == text[i-1] and count < 9:
 count += 1
 else:
 result.append(f"{count}{text[i-1]}")
 count = 1
result.append(f"{count}{text[-1]}")
return "".join(result)
...

```

##### **(b) `decode(text)`**

- **Lösung:**

```

python
def decode(text):
 result = []
 i = 0
 while i < len(text):
 if text[i].isdigit():
 count = int(text[i])
 result.append(text[i+1] * count)
 i += 2
 else:
 result.append(text[i])
 i += 1
 return "".join(result)
...

```

##### **(c) RLE-Nachteil**

- **Antwort:** Wenn der Text viele verschiedene Zeichen hat (z. B. `ABCABCABC`), wird die Länge dur

##### **(d) `checkRLE(text)`**

- **Lösung:**

```

python
def checkRLE(text):
 return len(encode(text)) < len(text)
...

```

---

### **Aufgabe 4: Schimpfwortfilter**

##### **(a) `readSwearwordData(filename)`**

- **Lösung:**

```

python
def readSwearwordData(filename):
 try:
 with open(filename, "r") as file:
 return [line.strip().split() for line in file]
 except:
 return []
...

```

```
** (b) `checkText(message, swearwords)`**
```

```
- **Lösung:**
```

```
```python
```

```
def checkText(message, swearwords):
```

```
    total_severity = 0
```

```
    words = message.split()
```

```
    for i, word in enumerate(words):
```

```
        for entry in swearwords:
```

```
            if word.lower() == entry[0].lower():
```

```
                total_severity += entry[1]
```

```
                words[i] = "***"
```

```
    if total_severity > 7:
```

```
        return "Nachricht wurde gelöscht"
```

```
    return " ".join(words)
```

```
```
```

```
** (c) Hauptprogramm**
```

```
- **Lösung:**
```

```
```python
```

```
print(checkText("Du bist natürlich kein Idiot", readSwearwordData("swearwords.txt")))
```

```
```
```

---

```
** Aufgabe 5: Java-Klasse `Moebel`**
```

```
- **Lösung:**
```

```
```java
```

```
public class Moebel {
```

```
    private String name;
```

```
    private double preis;
```

```
    public Moebel(String name, double preis) {
```

```
        this.name = name;
```

```
        this.preis = preis;
```

```
    }
```

```
    @Override
```

```
    public String toString() {
```

```
        return name + " (" + preis + " €)";
```

```
    }
```

```
}
```

```
```
```

---

```
Zusammenfassung:
```

- **\*\*Fehleranalyse:\*\*** Typische Python-Fehler (Groß-/Kleinschreibung, uninitialisierte Variablen, Division)
- **\*\*RLE:\*\*** Komprimierung mit Begrenzung auf 9 Wiederholungen.
- **\*\*Schimpfwortfilter:\*\*** Einlesen von Wörtern, Zensierung und Schweregradprüfung.

- **Java:** Einfache Klasse mit Konstruktor und `toString()`.